

Milestone 5 – JavaScript & API Cheatsheet

♦ var vs let vs const

- **var**
 - 👉 Function scoped, block respect করে না
 - 👉 Hoisting এর কারণে unexpected bug হয়
- **let**
 - 👉 Block scoped {}
 - 👉 Value change করা যায়
- **const**
 - 👉 Block scoped
 - 👉 Value reassign করা যায় না (best practice)

 Recommendation: **let & const** ব্যবহার করো

♦ Default Parameter

```
function greet(name = "Guest") {  
  return `Hello ${name}`;  
}
```

- 👉 Function call করার সময় argument না দিলে default value নেয়
 - 👉 Error-free function লেখার জন্য খুব দরকারি
-

♦ Template String

```
`My name is ${name}`
```

- 👉 `${}` দিয়ে dynamic value বসানো যায়
- 👉 Multiline string লেখা যায়
- 👉 UI text বানাতে খুব useful

♦ Arrow Function

```
const add = (a, b) => a + b;
```

- 👉 Short syntax
- 👉 Clean code
- 👉 Own `this` নেই (important for React & modern JS)

♦ Spread Operator (...)

```
const newArr = [...arr];  
const max = Math.max(...arr);
```

- 👉 Array copy করা যায়
- 👉 Multiple array merge করা যায়
- 👉 Function এ value pass করা সহজ হয়

♦ Object & Array Destructuring

```
const {name, age} = user;  
const [a, b] = numbers;
```

- 👉 Object/array থেকে direct value বের করা যায়
- 👉 Code ছোট ও readable হয়

♦ Object Utilities

```
Object.keys(obj)  
Object.values(obj)  
Object.entries(obj)
```

- 👉 Object এর key, value loop বা analyze করার জন্য

- `delete obj.key` → property remove
 - `Object.seal()` → structure lock
 - `Object.freeze()` → full lock (no change)
-

♦ **Optional Chaining**

`user?.address?.city`

- 👉 Nested object এ value না থাকলেও error দিবে না
 - 👉 API data handle করার সময় খুব important
-

♦ **Object Loop**

`for (let key in obj) {}`

`for (let [k, v] of Object.entries(obj)) {}`

- 👉 Object এর ভিতরের সব data access করা যায়
-

♦ **Primitive vs Non-Primitive**

- **Primitive** → number, string, boolean, null, undefined
 - 👉 Value copy হয়
 - **Non-Primitive** → object, array, function
 - 👉 Reference copy হয়
-

♦ **null vs undefined**

- **undefined**
👉 Value assign করা হয়নি
- **null**
👉 Intentionally empty রাখা হয়েছে

📌 Interview favourite question

♦ Truthy & Falsy

Falsy values:

false, 0, "", null, undefined, NaN

- 👉 এগুলো condition এ false behave করে
 - 👉 Bug avoid করতে জানা জরুরি
-

♦ == VS ===

- == → Type convert করে check করে
- === → Type + Value দুটোই check করে

📌 Always use ===

♦ Scope

- **Global Scope** → সব জায়গা থেকে access
- **Block Scope** → {} এর ভিতর
- **Function Scope** → function এর ভিতর

♦ Hoisting

👉 JS variable/function memory তে আগে রাখে

👉 `var` hoisting এর কারণে bug হয়

♦ Closure

👉 Inner function outer function এর variable মনে রাখে

👉 Data privacy & advanced logic এ ব্যবহার হয়

♦ Callback Function

```
function main(fn) {  
  fn();  
}
```

👉 Function কে argument হিসেবে পাঠানো

👉 Async JS এর base concept

♦ Pass by Value vs Pass by Reference

- Primitive → value copy হয়
 - Object/Array → memory reference share হয়
-

♦ Increment / Decrement

`++i` // আগে বাড়ায়

`i++` // পরে বাড়ায়

👉 Loop ও calculation এ গুরুত্বপূর্ণ

♦ Array Power Methods

```
map()    // transform  
filter() // condition  
find()   // single item  
reduce() // calculation  
forEach() // loop
```

👉 Real-world data handling এর backbone

♦ API Basics

👉 API হলো server থেকে data আনার রাস্তা

👉 HTTP vs HTTPS → HTTPS secure

♦ JSON

```
JSON.parse()    // string → object  
JSON.stringify() // object → string
```

👉 API data handle করতে লাগে

♦ Fetch API

```
fetch(url)  
  .then(res => res.json())
```

👉 Server থেকে data আনার standard way

♦ HTTP Methods

- GET → data পড়া
- POST → নতুন data
- PATCH → update
- DELETE → remove

👉 CRUD concept

♦ Debug API

👉 Network tab

👉 Status code (200 OK, 404 Not Found, 500 Server Error)

👉 Headers check

👉 তুমি পারবে—

- API থেকে data load
 - Spinner show/hide
 - Search implement
 - Empty state handle
 - Modal open/close
 - Pronounce feature add
-

♦ Async / Await

```
async function loadData() {  
  const res = await fetch(url);  
  const data = await res.json();  
}
```

}

- 👉 Promise সহজভাবে handle করা
 - 👉 Clean & readable async code
-